SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

THANIGAIVEL S
192565006D

CSA0620_DAA_CO1_AT1

← Back

Language: Python | C | C++ | Java

QUESTIONS

- **Q1** Master Theorem: Four Subproblems — 4 marks
- **Q2** Master Theorem: Unequal Growth — 4 marks
- **Q3** Master Theorem: Square-Root Work — 4 marks
- **Q4** Master Theorem: Five Subproblems — 4 marks
- **Q5** Master Theorem: Logarithmic Factor — 4 marks
- **Q6** Substitution: Linearithmic Recurrence

Q1 Master Theorem: Four Subproblems — 4 marks

Analyze $T(n) = 4T(n/2) + n^2 \log n$ using the Master Theorem. Identify a, b, and f(n), state whether a standard case applies, and justify the asymptotic bound.

Your Answer

Given: $T(n) = 4T(n/2) + n^2 \log n$

Here, $a = 4$, $b = 2$, so $f(n) = n^2 \log n$.

Step 1: Find n^(log_b a)
$\log_b a = \log_2 4 = 2$

Save Answer | Next →

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

THANIGAIVEL S
192565006D

CSA0620_DAA_CO1_AT1

← Back

Language: Python | C | C++ | Java

QUESTIONS

- **Q1** Master Theorem: Four Subproblems — 4 marks
- **Q2** Master Theorem: Unequal Growth — 4 marks
- **Q3** Master Theorem: Square-Root Work — 4 marks
- **Q4** Master Theorem: Five Subproblems — 4 marks
- **Q5** Master Theorem: Logarithmic Factor — 4 marks
- **Q6** Substitution: Linearithmic Recurrence

Q5 Master Theorem: Logarithmic Factor — 4 marks

Analyze T(n) = 16T(n/4) + n²/log n. Discuss whether the standard Master Theorem applies directly and provide a justified bound.

Your Answer

To analyze T(n)= 16T(n/4)+ (n^2)/(log n):

1. Identify parameters: a = 16, b = 4, and f(n)= (n^2)/(log n).
2. Calculate n^(log_b a): n^(log_4 16) = n^2.
3. Compare f(n) with n^(log_b a): We are comparing f(n)= (n^2)/(log n) with n^2.
4. Applicability of Standard Master Theorem: The standard Master Theorem (specifically Case 2) requires f(n)= Θ(n^(log_b

Save Answer | Next →

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

THANIGAIVEL S
192565006D

CSA0620_DAA_CO1_AT1

← Back

Language: Python | C | C++ | Java

QUESTIONS

- **Q1** Master Theorem: Four Subproblems — 4 marks
- **Q2** Master Theorem: Unequal Growth — 4 marks
- **Q3** Master Theorem: Square-Root Work — 4 marks
- **Q4** Master Theorem: Five Subproblems — 4 marks
- **Q5** Master Theorem: Logarithmic Factor — 4 marks
- **Q6** Substitution: Linearithmic Recurrence

Q6 Substitution: Linearithmic Recurrence — 4 marks

Use the substitution method to prove an asymptotic bound for $T(n) = 2T(n/2) + n$. State an induction hypothesis and base case.

Your Answer

Induction Hypothesis:
Assume that $T(k) \leq c \cdot k \log k$ for all $k < n$, where c is a positive constant.

Base Case:
For a small constant n, say n = 2:
$T(2) = 2T(1) + 2$.

Save Answer | Next →

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

THANIGAIVEL S
192565006D

CSA0620_DAA_CO1_AT1

← Back

Language: Python | C | C++ | Java

QUESTIONS

- **Q1** Master Theorem: Four Subproblems — 4 marks
- **Q2** Master Theorem: Unequal Growth — 4 marks
- **Q3** Master Theorem: Square-Root Work — 4 marks
- **Q4** Master Theorem: Five Subproblems — 4 marks
- **Q5** Master Theorem: Logarithmic Factor — 4 marks
- **Q6** Substitution: Linearithmic Recurrence

Q7 Substitution: Decreasing by One — 4 marks

Use substitution or expansion to establish a tight asymptotic bound for T(n) = T(n−1) + n, assuming T(1)=1.

Your Answer

To find the tight asymptotic bound for T(n)= T(n − 1)+ n with T(1)= 1, we can use the method of expansion (iteration):

1. T(n)= T(n − 1)+ n
2. Substitute T(n − 1): T(n)=[T(n − 2)+(n − 1)]+ n
3. Substitute T(n − 2): T(n)=[T(n − 3)+(n − 2)]+(n − 1)+ n
4. Following this pattern to the base case T(1): T(n)= 1 + 2 + 3 + ... + n

Save Answer | Next →

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

THANIGAIVEL S
192565006D

CSA0620_DAA_CO1_AT1

← Back

Language: Python | C | C++ | Java

QUESTIONS

- **Q1** Master Theorem: Four Subproblems — 4 marks
- **Q2** Master Theorem: Unequal Growth — 4 marks
- **Q3** Master Theorem: Square-Root Work — 4 marks
- **Q4** Master Theorem: Five Subproblems — 4 marks
- **Q5** Master Theorem: Logarithmic Factor — 4 marks
- **Q6** Substitution: Linearithmic Recurrence

Q8 Substitution: Halving with Constant Work — 4 marks

Prove that T(n) = T(n/2) + 1 is Θ(log n), assuming n is a power of two.

Your Answer

To prove T(n)= T(n/2)+ 1 is Θ(log n) for n = 2^k:

1. Base Case: Let n = 2^0 = 1. Then T(1) is a constant c. log_2 (1)= 0, so the base case holds.

2. Substitution/Iteration Method:

Save Answer | Next →

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

THANIGAIVEL S
192565006D

CSA0620_DAA_CO1_AT1

← Back

Language: Python | C | C++ | Java

QUESTIONS

- **Q1** Master Theorem: Four Subproblems — 4 marks
- **Q2** Master Theorem: Unequal Growth — 4 marks
- **Q3** Master Theorem: Square-Root Work — 4 marks
- **Q4** Master Theorem: Five Subproblems — 4 marks
- **Q5** Master Theorem: Logarithmic Factor — 4 marks
- **Q6** Substitution: Linearithmic Recurrence

Q9 Substitution: Exponential Recurrence — 4 marks

Establish a tight asymptotic bound for T(n) = 2T(n−1) + 1 with T(0)=1. Explain the growth pattern.

Your Answer

To find the tight asymptotic bound for T(n)= 2T(n − 1)+ 1 with T(0)= 1, we can use the substitution method (unrolling the recurrence):

1. T(n)= 2T(n − 1)+ 1
2. T(n)= 2 [2T(n − 2)+ 1]+ 1 = 2^2T(n − 2)+ 2 + 1
3. T(n)= 2^2 [2T(n − 3)+ 1]+ 2 + 1 = 2^3T(n − 3)+ 2^2 + 2^1 + 2^0
4. Following the pattern for k steps: T(n)= 2^kT(n − k)+∑_(i=0)^(k−1)2^i

Save Answer | Next →

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

THANI
1925650

Screenshot taken
View image

CSA0620_DAA_CO1_AT1

← Back

Language: Python | C | C++ | Java

QUESTIONS

- **Q1** Master Theorem: Four Subproblems — 4 marks
- **Q2** Master Theorem: Unequal Growth — 4 marks
- **Q3** Master Theorem: Square-Root Work — 4 marks
- **Q4** Master Theorem: Five Subproblems — 4 marks
- **Q5** Master Theorem: Logarithmic Factor — 4 marks
- **Q6** Substitution: Linearithmic Recurrence

Q10 Check a Complexity Claim — 4 marks

An analyst claims T(n) = T(n/2) + n is Θ(n log n). Assess the claim and establish the correct asymptotic bound.

Your Answer

The claim that T(n)= T(n/2)+ n is Θ(n log n) is incorrect.

Using the Master Theorem for the recurrence T(n)= aT(n/b)+ f(n):

* a = 1
* b = 2

Save Answer | Next →

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

THANIGAIVEL S
192565006D

Screenshot taken
View image

CSA0620_DAA_CO1_AT1

Language: Python | C | C++ | Java

QUESTIONS

- **Q1** Master Theorem: Four Subproblems — 4 marks
- **Q2** Master Theorem: Unequal Growth — 4 marks
- **Q3** Master Theorem: Square-Root Work — 4 marks
- **Q4** Master Theorem: Five Subproblems — 4 marks
- **Q5** Master Theorem: Logarithmic Factor — 4 marks
- **Q6** Substitution: Linearithmic Recurrence

Q11 Algorithm Crossover — 4 marks

Algorithm A performs $5n \log_2 n$ operations and Algorithm B performs $0.02n^2$ operations. Compare them and estimate the crossover input size.

Your Answer

To find the crossover point, we set the two equations equal to each other:
$5n \log_2 n = 0.02n^2$

Divide both sides by n (assuming $n > 0$):
$5 \log_2 n = 0.02n$

Save Answer | Next →

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

THANI
1925650

Screenshot taken
View image

CSA0620_DAA_CO1_AT1

← Back

Language: Python | C | C++ | Java

QUESTIONS

- **Q1** Master Theorem: Four Subproblems — 4 marks
- **Q2** Master Theorem: Unequal Growth — 4 marks
- **Q3** Master Theorem: Square-Root Work — 4 marks
- **Q4** Master Theorem: Five Subproblems — 4 marks
- **Q5** Master Theorem: Logarithmic Factor — 4 marks
- **Q6** Substitution: Linearithmic Recurrence

Q12 **Scalability: n log n** — 4 marks

A program processes 10,000 records in 2 seconds using an $O(n \log_2 n)$ algorithm. Estimate runtime for 100,000 records, assuming proportional scaling and negligible fixed overhead.

Your Answer

For Q12 Scalability: n log n, here is the calculation and the answer you should enter:

1. Initial state: n_1 = 10,000, T(n_1)= 2 seconds.
2. Algorithm: T(n)= k · (n log_2 n).
3. New state: n_2 = 100,000.
4. Ratio calculation:

Save Answer | Next →

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

THANIGAIVEL S
192565006D

Screenshot taken
View image

Screenshot taken
View image

CSA0620_DAA_CO1_AT1

Language: Python | C | C++ | Java

QUESTIONS

- **Q1** Master Theorem: Four Subproblems — 4 marks
- **Q2** Master Theorem: Unequal Growth — 4 marks
- **Q3** Master Theorem: Square-Root Work — 4 marks
- **Q4** Master Theorem: Five Subproblems — 4 marks
- **Q5** Master Theorem: Logarithmic Factor — 4 marks
- **Q6** Substitution: Linearithmic Recurrence

Q13 **Scalability: Quadratic** — 4 marks

An O(n²) algorithm takes 4 seconds for 2,000 items. Estimate runtime for 6,000 items, assuming the quadratic term dominates.

Your Answer

Calculation breakdown:

1. The number of items increases from 2,000 to 6,000 (a factor of 3).
2. Since the algorithm is O(n^2), the runtime increases by the square of that factor.
3. 3^2 = 9.
4. 4 seconds × 9 = 36 seconds.

Save Answer | Next →

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

THANIGAIVEL S
192565006D

CSA0620_DAA_CO1_AT1

← Back

Language: Python | C | C++ | Java

QUESTIONS

- **Q1** Master Theorem: Four Subproblems — 4 marks
- **Q2** Master Theorem: Unequal Growth — 4 marks
- **Q3** Master Theorem: Square-Root Work — 4 marks
- **Q4** Master Theorem: Five Subproblems — 4 marks
- **Q5** Master Theorem: Logarithmic Factor — 4 marks
- **Q6** Substitution: Linearithmic Recurrence

Q14 Compare Three Growth Rates — 4 marks

Compare O(n), O(n log n), and O(n^2) growth. Discuss how their operation counts scale and give an appropriate use case for each.

Your Answer

O(n), O(n log n), and O(n^2) represent different levels of computational complexity:

1. O(n) - Linear Growth: The operation count increases proportionally with the input size. If the input doubles, the time taken roughly doubles.

Save Answer | Next →

Screenshot taken — View image

Screenshot taken — View image

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

THANI
1925650

Screenshot taken
View image

Screenshot taken
View image

CSA0620_DAA_CO1_AT1

← Back

Language: Python | C | C++ | Java

QUESTIONS

Q1 ✓
Master Theorem: Four Subproblems
4 marks

Q2 ✓
Master Theorem: Unequal Growth
4 marks

Q3 ✓
Master Theorem: Square-Root Work
4 marks

Q4 ✓
Master Theorem: Five Subproblems
4 marks

Q5 ✓
Master Theorem: Logarithmic Factor
4 marks

Q6 ✓
Substitution: Linearithmic Recurrence

Q15 **Constant Factors and Big-O** 4 marks

Two algorithms take $T_1(n)=100n$ and $T_2(n)=n \log_2 n$. Explain why asymptotic notation alone may not determine which is faster for small inputs.

Your Answer

Asymptotic notation (like Big-O) describes how an algorithm performs as the input size (n) approaches infinity, focusing on the growth rate rather than exact values. However, for small inputs, the constant factors matter significantly. In this case, T_1(n)= 100n has a large constant factor of 100, while T_2(n)= n log_2 n has a much smaller coefficient. For small values of n, the 100n term will produce a much larger result than n log_2 n, making T_2 faster in practice until n becomes large enough for the logarithmic growth to be overshadowed by the constant.

Save Answer | Next →

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

THANI
1925650

Screenshot taken
View image

Screenshot taken
View image

CSA0620_DAA_CO1_AT1

Language: Python | C | C++ | Java

QUESTIONS

Q1 ✓
Master Theorem: Four Subproblems
4 marks

Q2 ✓
Master Theorem: Unequal Growth
4 marks

Q3 ✓
Master Theorem: Square-Root Work
4 marks

Q4 ✓
Master Theorem: Five Subproblems
4 marks

Q5 ✓
Master Theorem: Logarithmic Factor
4 marks

Q6 ✓
Substitution: Linearithmic Recurrence

Q16 Hash Set: Time and Space — 4 marks

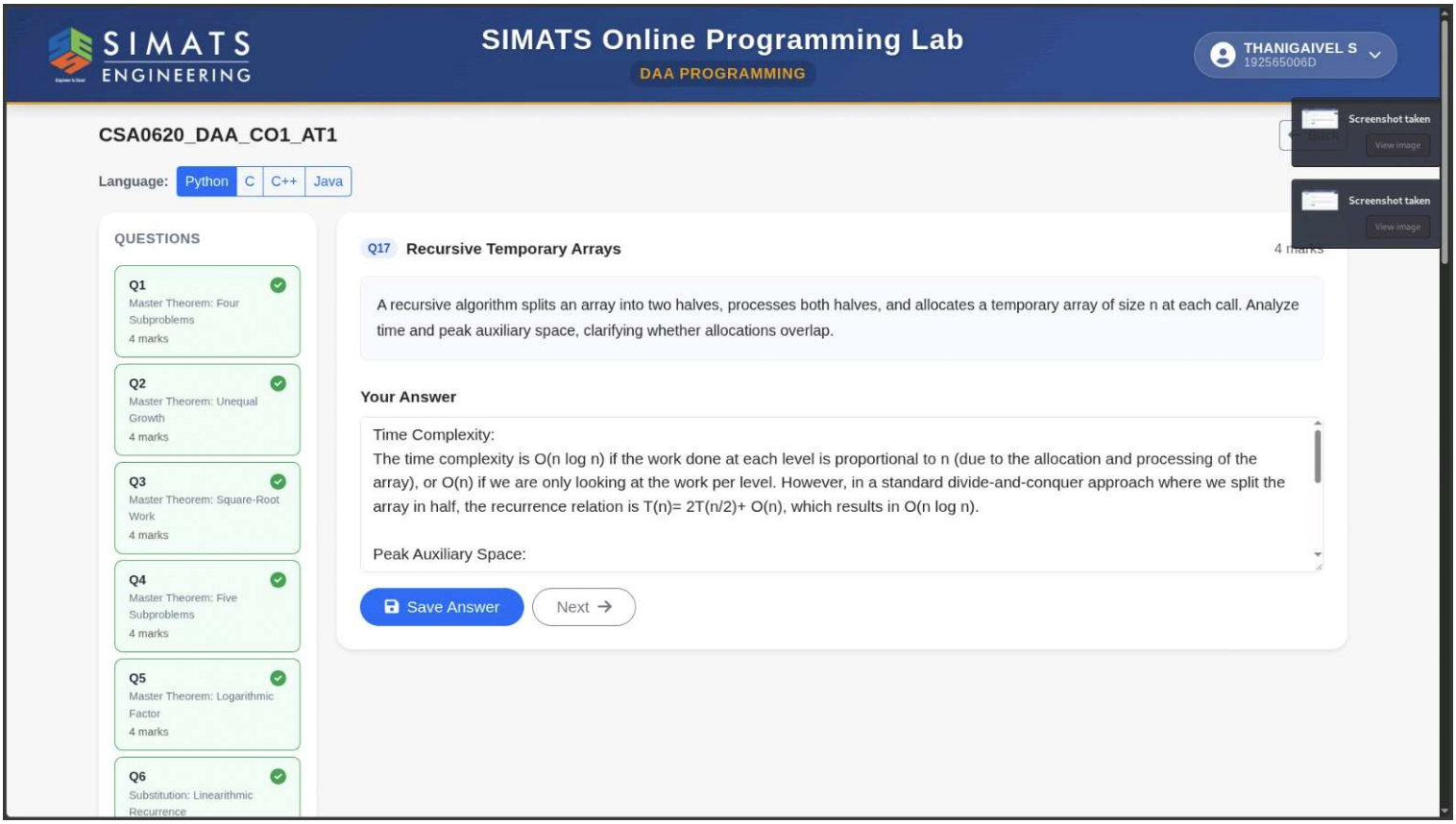
An algorithm scans an array once and stores distinct values in a hash set. Analyze expected time and auxiliary space, stating assumptions about hash operations.

Your Answer

Time Complexity: The expected time complexity is O(n). This is because the algorithm iterates through the array of size n exactly once. Assuming that hash set operations (insert and lookup) take O(1) on average, the total time spent is n × O(1)= O(n).

Auxiliary Space Complexity: The expected auxiliary space complexity is O(k), where k is the number of distinct elements stored in the hash set. In the worst-case scenario where all elements in the array are unique, the space complexity becomes O(n).

Save Answer | Next →


SIMATS ENGINEERING
SIMATS Online Programming Lab
DAA PROGRAMMING
THANIGAIVEL S
192565006D
CSA0620_DAA_CO1_AT1
Language: Python C C++ Java
QUESTIONS
Q1
Master Theorem: Four Subproblems
4 marks
Q2
Master Theorem: Unequal Growth
4 marks
Q3
Master Theorem: Square-Root Work
4 marks
Q4
Master Theorem: Five Subproblems
4 marks
Q5
Master Theorem: Logarithmic Factor
4 marks
Q6
Substitution: Linearithmic Recurrence
Q17 Recursive Temporary Arrays
4 marks
A recursive algorithm splits an array into two halves, processes both halves, and allocates a temporary array of size n at each call. Analyze time and peak auxiliary space, clarifying whether allocations overlap.
Your Answer
Time Complexity:
The time complexity is O(n log n) if the work done at each level is proportional to n (due to the allocation and processing of the array), or O(n) if we are only looking at the work per level. However, in a standard divide-and-conquer approach where we split the array in half, the recurrence relation is T(n)= 2T(n/2)+ O(n), which results in O(n log n).
Peak Auxiliary Space:
Save Answer
Next
Screenshot taken
View image
Screenshot taken
View image

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

THANI
1925650

Screenshot taken
View image

Screenshot taken
View image

CSA0620_DAA_CO1_AT1

← Back

Language: Python | C | C++ | Java

QUESTIONS

- **Q1** Master Theorem: Four Subproblems — 4 marks
- **Q2** Master Theorem: Unequal Growth — 4 marks
- **Q3** Master Theorem: Square-Root Work — 4 marks
- **Q4** Master Theorem: Five Subproblems — 4 marks
- **Q5** Master Theorem: Logarithmic Factor — 4 marks
- **Q6** Substitution: Linearithmic Recurrence

Q18 **Nested Loops with a Condition** — 4 marks

An algorithm has three loops from 1 to n, but the innermost statement executes only when i+j≤n. Determine its time complexity and justify the count.

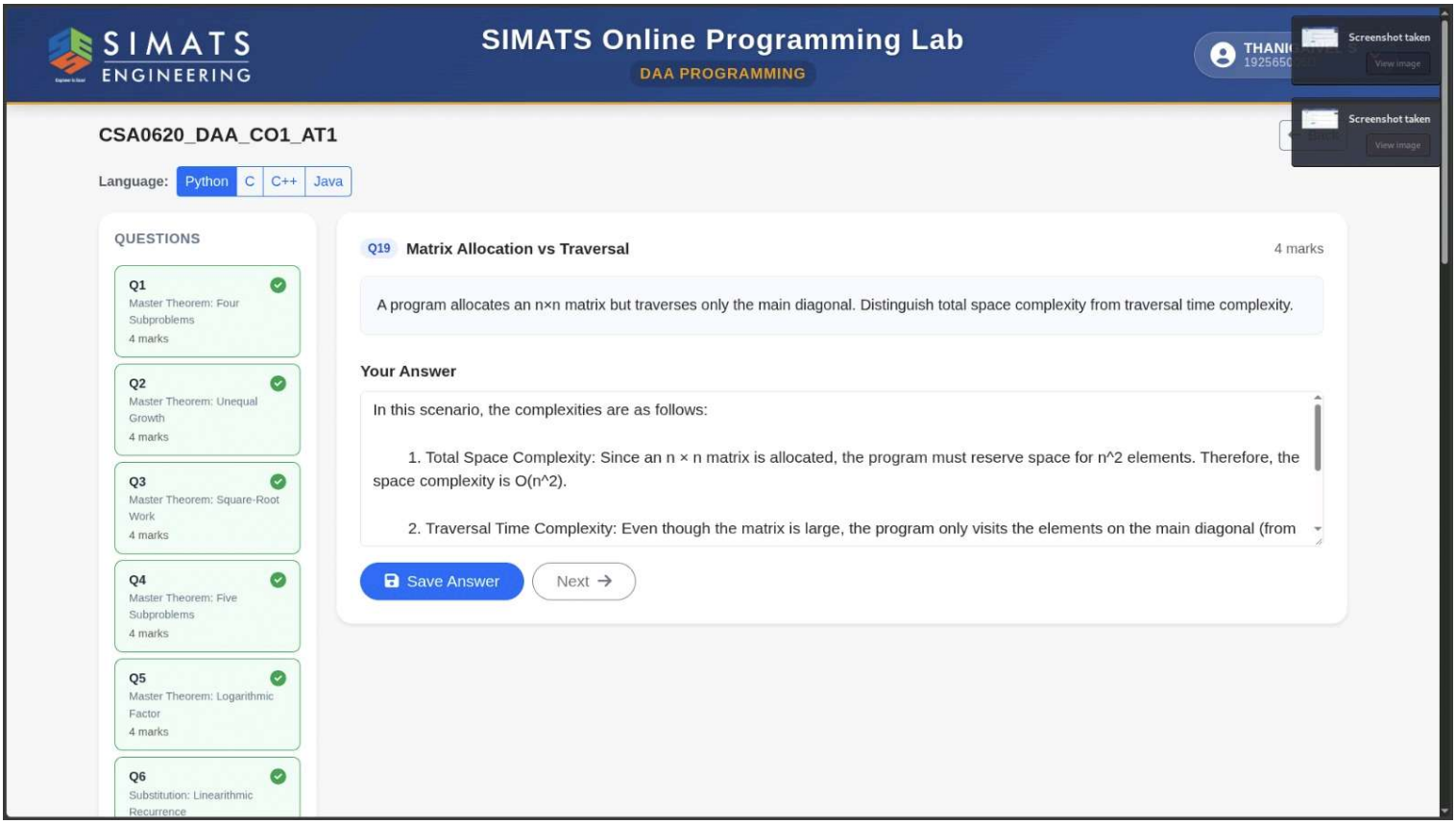
Your Answer

The time complexity is Θ(n^2).

Justification:
The algorithm has three nested loops. However, the condition i + j ≤ n restricts the execution of the innermost loop.

1. The outermost loop runs n times.

Save Answer | Next →


SIMATS ENGINEERING
SIMATS Online Programming Lab
DAA PROGRAMMING
Screenshot taken
View image
Screenshot taken
View image
CSA0620_DAA_CO1_AT1
Language: Python C C++ Java
QUESTIONS
Q1
Master Theorem: Four Subproblems
4 marks
Q2
Master Theorem: Unequal Growth
4 marks
Q3
Master Theorem: Square-Root Work
4 marks
Q4
Master Theorem: Five Subproblems
4 marks
Q5
Master Theorem: Logarithmic Factor
4 marks
Q6
Substitution: Linearithmic Recurrence
Q19 Matrix Allocation vs Traversal
4 marks
A program allocates an n×n matrix but traverses only the main diagonal. Distinguish total space complexity from traversal time complexity.
Your Answer
In this scenario, the complexities are as follows:
1. Total Space Complexity: Since an n × n matrix is allocated, the program must reserve space for n^2 elements. Therefore, the space complexity is O(n^2).
2. Traversal Time Complexity: Even though the matrix is large, the program only visits the elements on the main diagonal (from
Save Answer
Next

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

THANIGAIVEL S
192565006D

CSA0620_DAA_CO1_AT1

← Back

Language: Python | C | C++ | Java

Screenshot taken
View image

QUESTIONS

- **Q1** ✓ Master Theorem: Four Subproblems — 4 marks
- **Q2** ✓ Master Theorem: Unequal Growth — 4 marks
- **Q3** ✓ Master Theorem: Square-Root Work — 4 marks
- **Q4** ✓ Master Theorem: Five Subproblems — 4 marks
- **Q5** ✓ Master Theorem: Logarithmic Factor — 4 marks
- **Q6** ✓ Substitution: Linearithmic Recurrence

Q20 **Binary Search: Time and Space** — 4 marks

Compare iterative and recursive binary search in time and auxiliary space. Explain the role of recursion-stack overhead.

Your Answer

Iterative Binary Search:

* Time Complexity: O(log n) because the search space is halved in each step.
* Auxiliary Space: O(1) because it only uses a few variables (like low, high, and mid) regardless of the input size.

Recursive Binary Search:

Save Answer | Next →

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

THANI
1925650

Screenshot taken
View image

CSA0620_DAA_CO1_AT1

← Back

Language: Python | C | C++ | Java

QUESTIONS

- **Q1** Master Theorem: Four Subproblems — 4 marks
- **Q2** Master Theorem: Unequal Growth — 4 marks
- **Q3** Master Theorem: Square-Root Work — 4 marks
- **Q4** Master Theorem: Five Subproblems — 4 marks
- **Q5** Master Theorem: Logarithmic Factor — 4 marks
- **Q6** Substitution: Linearithmic Recurrence

Q21 **Unequal Subproblem Sizes** — 4 marks

Analyze T(n)=T(n/3)+T(2n/3)+n using a recursion-tree argument. Explain the work per level and total tree depth.

Your Answer

Work per Level:
At the root (Level 0), the work done is n.
At Level 1, the subproblems are n/3 and 2n/3. Summing these gives: (n/3)+(2n/3)= n.
At Level 2, the subproblems are n/9, 2n/9, 2n/9, and 4n/9. Summing these gives: (n/9 + 2n/9 + 2n/9 + 4n/9)= n.
In this specific recurrence, the total work performed at each full level of the tree remains exactly n.

Save Answer | Next →

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

THANI
1925650

Screenshot taken
View image

Screenshot taken
View image

CSA0620_DAA_CO1_AT1

Language: Python | C | C++ | Java

QUESTIONS

Q1 ✓
Master Theorem: Four Subproblems
4 marks

Q2 ✓
Master Theorem: Unequal Growth
4 marks

Q3 ✓
Master Theorem: Square-Root Work
4 marks

Q4 ✓
Master Theorem: Five Subproblems
4 marks

Q5 ✓
Master Theorem: Logarithmic Factor
4 marks

Q6 ✓
Substitution: Linearithmic Recurrence

Q22 **Unequal Split with Linear Combine** 4 marks

A divide-and-conquer algorithm splits into subproblems of sizes n/4 and 3n/4 and performs Θ(n) combining work. Determine its asymptotic time complexity using a suitable method.

Your Answer

The recurrence relation for this algorithm is:
T(n) = T(n/4) + T(3n/4) + Θ(n)

To solve this, we can use the Recursion Tree method:

1. At the root, the work is n.

Save Answer | Next →

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

THANIGAIVEL S
192565006D

Screenshot taken
View image

Screenshot taken
View image

CSA0620_DAA_CO1_AT1

Language: Python | C | C++ | Java

QUESTIONS

- **Q1** Master Theorem: Four Subproblems — 4 marks
- **Q2** Master Theorem: Unequal Growth — 4 marks
- **Q3** Master Theorem: Square-Root Work — 4 marks
- **Q4** Master Theorem: Five Subproblems — 4 marks
- **Q5** Master Theorem: Logarithmic Factor — 4 marks
- **Q6** Substitution: Linearithmic Recurrence

Q23 Fibonacci-Style Recurrence — 4 marks

A recursive procedure satisfies $T(n)=T(n-1)+T(n-2)+\Theta(1)$. Determine its asymptotic growth and explain the source of the growth.

Your Answer

The recurrence relation T(n)= T(n − 1)+ T(n − 2)+ Θ(1) follows the same growth pattern as the Fibonacci sequence. The asymptotic growth is O(ϕ^n), where ϕ (the golden ratio) is approximately 1.618. The source of this exponential growth is the branching nature of the recursion tree, where each call generates two additional recursive calls, leading to a tree depth of n and a total number of nodes that grows exponentially with n.

Save Answer | Next →

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

THANI
1925650

Screenshot taken
View image

Screenshot taken
View image

CSA0620_DAA_CO1_AT1

← Back

Language: Python | C | C++ | Java

QUESTIONS

Q1 ✓
Master Theorem: Four Subproblems
4 marks

Q2 ✓
Master Theorem: Unequal Growth
4 marks

Q3 ✓
Master Theorem: Square-Root Work
4 marks

Q4 ✓
Master Theorem: Five Subproblems
4 marks

Q5 ✓
Master Theorem: Logarithmic Factor
4 marks

Q6 ✓
Substitution: Linearithmic Recurrence

Q24 **Four Half-Size Subproblems** 4 marks

Design and analyze a recurrence for an algorithm that splits into four subproblems of size n/2 and spends Θ(n) combining their results.

Your Answer

The recurrence relation is:
T(n) = 4T(n/2) + Θ(n)

Using the Master Theorem:
a = 4, b = 2, f(n) = Θ(n)
Compare f(n) with n^(log_b(a)):

Save Answer | Next →

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

THANIGAIVEL S
192565006D

CSA0620_DAA_CO1_AT1

← Back

Language: Python | C | C++ | Java

QUESTIONS

- **Q1** Master Theorem: Four Subproblems — 4 marks
- **Q2** Master Theorem: Unequal Growth — 4 marks
- **Q3** Master Theorem: Square-Root Work — 4 marks
- **Q4** Master Theorem: Five Subproblems — 4 marks
- **Q5** Master Theorem: Logarithmic Factor — 4 marks
- **Q6** Substitution: Linearithmic Recurrence

Q2 Master Theorem: Unequal Growth — 4 marks

Analyze T(n) = 3T(n/2) + n using the Master Theorem, where applicable. Explain which term dominates.

Your Answer

T(n) = 3T(n/2) + n

Using the Master Theorem:
Here, a = 3, b = 2, and f(n) = n.

Step 1: Find n^(log_b a)

Save Answer | Next →

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

THANIGAIVEL S
192565006D

CSA0620_DAA_CO1_AT1

← Back

Language: Python | C | C++ | Java

QUESTIONS

- **Q1** Master Theorem: Four Subproblems — 4 marks
- **Q2** Master Theorem: Unequal Growth — 4 marks
- **Q3** Master Theorem: Square-Root Work — 4 marks
- **Q4** Master Theorem: Five Subproblems — 4 marks
- **Q5** Master Theorem: Logarithmic Factor — 4 marks
- **Q6** Substitution: Linearithmic Recurrence

Q3 Master Theorem: Square-Root Work — 4 marks

Find the asymptotic complexity of $T(n) = 2T(n/4) + \sqrt{n}$. State the parameters and justify the applicable case.

Your Answer

Given: $T(n) = 2T(n/4) + \sqrt{n}$

Parameters (comparing with $T(n) = aT(n/b) + f(n)$):

* $a = 2$
* $b = 4$

Save Answer | Next →

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

THANIGAIVEL S
192565006D

CSA0620_DAA_CO1_AT1

← Back

Language: Python | C | C++ | Java

QUESTIONS

- **Q1** Master Theorem: Four Subproblems — 4 marks
- **Q2** Master Theorem: Unequal Growth — 4 marks
- **Q3** Master Theorem: Square-Root Work — 4 marks
- **Q4** Master Theorem: Five Subproblems — 4 marks
- **Q5** Master Theorem: Logarithmic Factor — 4 marks
- **Q6** Substitution: Linearithmic Recurrence

Q4 Master Theorem: Five Subproblems — 4 marks

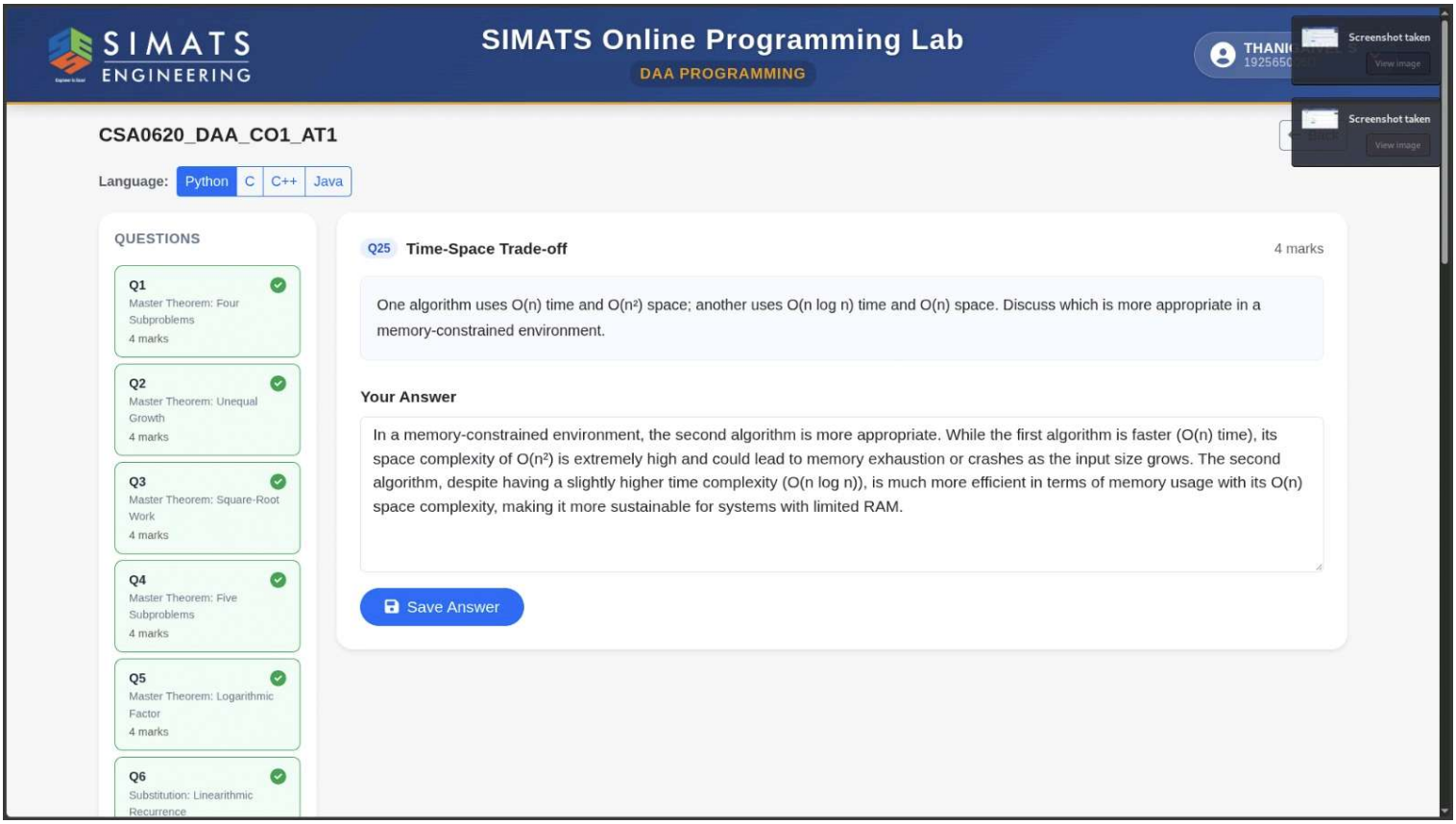
A divide-and-conquer algorithm makes 5 recursive calls on subproblems of size n/3 and performs $\Theta(n^2)$ extra work. Determine its asymptotic time complexity.

Your Answer

Based on the Master Theorem formula T(n)= aT(n/b)+ f(n):

* a = 5 (number of subproblems)
* b = 3 (size reduction factor)
* f(n)= Θ(n^2) (extra work)

Save Answer | Next →


SIMATS ENGINEERING
SIMATS Online Programming Lab
DAA PROGRAMMING
Screenshot taken
View image
Screenshot taken
View image
CSA0620_DAA_CO1_AT1
Language: Python C C++ Java
QUESTIONS
Q1
Master Theorem: Four Subproblems
4 marks
Q2
Master Theorem: Unequal Growth
4 marks
Q3
Master Theorem: Square-Root Work
4 marks
Q4
Master Theorem: Five Subproblems
4 marks
Q5
Master Theorem: Logarithmic Factor
4 marks
Q6
Substitution: Linearithmic Recurrence
Q25 Time-Space Trade-off
4 marks
One algorithm uses O(n) time and O(n²) space; another uses O(n log n) time and O(n) space. Discuss which is more appropriate in a memory-constrained environment.
Your Answer
In a memory-constrained environment, the second algorithm is more appropriate. While the first algorithm is faster (O(n) time), its space complexity of O(n²) is extremely high and could lead to memory exhaustion or crashes as the input size grows. The second algorithm, despite having a slightly higher time complexity (O(n log n)), is much more efficient in terms of memory usage with its O(n) space complexity, making it more sustainable for systems with limited RAM.
Save Answer